TEHNIČKA ŠKOLA RUĐER BOŠKOVIĆ

GETALDIĆEVA 4, ZAGREB

Marko Sikirić

Razvoj web stranice Interaktivna mapa u HTML-u, CSS-u i JavaScriptu

ZAVRŠNI RAD

ZAGREB, 2026.

TEHNIČKA ŠKOLA RUĐER BOŠKOVIĆ

GETALDIĆEVA 4, ZAGREB

Marko Sikirić

4.c

Tehničar za računalstvo

Razvoj web stranice Interaktivna mapa u HTML-u, CSS-u i JavaScriptu

ZAVRŠNI RAD

prof. Ivan Šaban

Zagreb, ožujak 2026

Sadržaj

Sadržaj iii
1. Uvod 1
2. Metode i tehnike rada 2
2.1. HTML – HyperText Markup Language 2
2.1.1. Općenito o HTML-u 2
2.1.2. Sintaksa i struktura HTML-a 3
2.2. CSS – Cascading Style Sheets 3
2.2.1. Općenito o CSS-u 3
2.2.2. Sintaksa i struktura CSS-a 4
2.3. JavaScript 4
2.3.1. Općenito o JavaScriptu 5
2.3.2. Sintaksa i struktura JavaScripta 5
2.3.3. JSON (JavaScript Object Notation) 6
2.4. Korišteni razvojni alati 7
2.4.1. Visual studio code 7
2.4.2. GitHub 7
2.4.3. Image Map Generator 8
3. Tehničko-tehnološki dio 11
3.1. Struktura projekta 11
3.1.1. Općenita struktura projekta 11
3.1.2. Glavna stranica s mapom 12
3.1.3. Datoteke zemalja 12
3.1.4. Stranica Kanonizacija (kanon.html) 13
3.1.4.1. Datoteka kalendar.json – baza podataka 14
3.1.5. Stranica O izradi (info.html) 15
3.1.6. Datoteka STIL.css – stilovi svih elemenata 15
3.1.6.1. Stil tijela stranice 15
3.1.6.2. Stil navigacijske trake 16
3.1.6.3. Kartica sveca 16

3.1.6.4. Podnožje (.detailed-footer) 17
3.1.7. Datoteka JAVA.js – JavaScript funkcionalnosti 18
3.1.7.1. Dinamički prikaz svetaca dana 18
3.1.7.2. Interaktivnost mape Europe 19
4. Zaključak 21
Popis literature 22

1. Uvod

Web stranca je ,u suvremenom svijetu, najbolji način za širenje vlastitih ideja i želja, a s time i Božje riječi i volje na internetu. Ona je neizostavan dio kada dolazi do širenja informacija i detaljno razrađenih tema putem interneta. Ovakav način širenja vlastitih ideja nije ograničen mjestom i jezikom te stranici može pristupiti bilo tko tko je motiviran za kršćanske teme i nauke.

Tijekom čitavog svog života sam usko povezan s Crkvom, pogotovo sa zajednicom mladih u kojoj trenutno imam ulogu animatora i organizatora. Isto tako sam aktivan u crkvenom zboru u kojem pjevam cijeli svoj život. Uglavnom Crkva je uvelike utjecala na moj život te je bila i glavni pokretač moje želje za izradom ovog završnog rada. Web stranica velikog i nedavno proglašenog svetog **Carla Acutisa** mi je bila glavna inspiracija za izradu, a on mi je osobno bio i uzor. Taj duh spajanja vjere i tehnologije temeljna je nit koja prožima cijeli projekt.

Tema moga završnog rada je **„Razvoj web stranice Interaktivna mapa u HTML-u, CSS-u i JavaScriptu“** te savršeno opisuje glavnu funkciju moje web stranice koja je svrstavanje katoličkih svetaca po državama Europe koristeći preko interaktivne mape. Ova web stranica je jedinstvena i jednostavna za korištenje, te je upravo zbog toga i važna, jer se razlikuje od drugih i donosi novi način prikaza povijesnih i crkvenih tema. Tema isto tako opisuje gradivo i navedene programske jezike koje sam usvojio tijekom svojeg školovanja u ovoj tehničkoj školi.

Moguće probleme koje bih mogao sresti upravo je označavanje država na interaktivnoj mapi i njihova respozivnost te postavljanje stranice online, da bude dostupna svim korisnicima diljem svijeta. Te probleme mogu riješiti korištenjem besplatnog **GitHub-a** za riješavanje porblema dostupnosti i korištenja domene te isto tako korištenjem besplatnog alata **Image Map Generatora** za označavanja površine država na slici mape Europe.

2. Metode i tehnike rada

U ovom poglavlju treba opisati koje će metode i tehnike biti korištene pri razradi teme, kako su provedene istraživačke aktivnosti, koji su programski alati ili aplikacije korišteni.

2.1. HTML – HyperText Markup Language

2.1.1. Općenito o HTML-u

HTML (HyperText Markup Language) je temeljni standard za strukturiranje sadržaja na internetu, a izumio ga je Tim Berners-Lee 1991. godine. Često svrstava u kategoriju računalnih jezika, ali važno je istaknuti da HTML nije programski jezik u klasičnom smislu jer on ne posjeduje logičke operatore, uvjete niti petlje, već primarno služi kao jezik za označavanje. Njegova je glavna uloga precizno opisivanje semantike i organizacije elemenata unutar dokumenta, koji preglednici zatim prikazuju kao vizualnu web stranicu. HyperText (nadtekst) označava tekst koji sadrži poveznice na druge dijelove istog dokumenta ili na potpuno druge dokumente, odnosno web stranice. Zahvaljujući HyperTextu, internet je živa cjelina povezana međusobno samo jednim klikom.

Ključni zaokret u evoluciji suvremenog weba pokrenula je radna skupina **WHATWG** (*Web Hypertext Application Technology Working Group*), koju su 2004. godine osnovale tvrtke Apple, Mozilla i Opera. Skupina se nastala kao reakcija na tadašnji spori razvoj standarda unutar konzorcija W3C, koji nije adekvatno odgovarao praktičnim potrebama web programera. Umjesto dotadašnjeg modela statičnih verzija, WHATWG je osnovao koncept „Living Standard" (živi standard). Prema tom modelu HTML specifikacije se kontinuirano analiziraju i usavršavaju te su u toku s tehnološkim napretkom, čime se izbjegavaju višegodišnja čekanja na formalna izdanja.

HTML5 ,predstavljen 2014., najnoviji je standard jezika za označavanje namijenjen strukturiranju i prezentaciji web sadržaja. Ovaj standard omogućuje bolju čitljivost koda i optimizaciju za tražilice, te je uveo podršku za audio i video podrške za multimedijske zapise, čime je u potpunosti maknuta potreba za vanjskim dodacima (engl. *plugins*). Upravo je HTML5 odabran kao tehnološki temelj ovog projekta, što je formalno naznačeno upotrebom <!DOCTYPE html> deklaracije na samom početku svake datoteke.

2.1.2. Sintaksa i struktura HTML-a

Strukturu HTML dokumenta sačinjavaju elementi definirani **oznakama** (engl. tags) unutar šiljastih zagrada. Većina se elemenata sastoji od parnih oznaka (npr. i) unutar kojih se smješta sadržaj. Nasuprot njima, određeni su elementi samozatvarajući (engl. self-closing), poput oznaka <img> za slike, <br> za prijelom retka ili <meta> za metapodatke, te oni ne sadrže ugniježđeni tekstualni sadržaj u svojim zagradama.

```
<title>Dobro došli!</title>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
```

Funkcionalnost elemenata dodatno se proširuje upotrebom **atributa** koji pružaju ključne informacije za interpretaciju ili oblikovanje sadržaja. Dok globalni atributi poput class i id služe za stiliziranje putem CSS-a i identifikaciju unutar skripti, specifični atributi poput href, src i alt definiraju odredišta poveznica, izvore multimedije i alternativne tekstove.

Hijerarhijska struktura HTML dokumenta naziva se **DOM** (Document Object Model). **DOM** služi kao programsko sučelje koje dokument reprezentira u obliku hijerarhijske strukture čvorova i objekata. Takav prikaz omogućuje programskim jezicima, prvenstveno JavaScriptu, izravnu interakciju sa stranicom radi dinamičke izmjene njezine strukture, vizualnog stila i samog sadržaja.

2.2. CSS – Cascading Style Sheets

2.2.1. Općenito o CSS-u

CSS predstavlja stilski jezik namijenjen oblikovanju HTML dokumenata, a njegov razvoj i standardizaciju predvodi konzorcij **W3C**. Dok je primarna uloga HTML-a definirati strukturu i semantiku sadržaja, CSS upravlja cjelokupnom vizualnom prezentacijom, uključujući aspekte poput tipografije, paleta boja, prostornog rasporeda elemenata i složenih animacija. Razdvajanje strukture dokumenta od njegova prikaza smatra se temeljom mrežnog razvoja, čime se postiže bolja pristupačnost te lakše održavanje kôda.

Naziv **kaskadni** (*cascading*) označava strogo definiran algoritam kojim preglednik rješava konflikte u situacijama kada više različitih CSS pravila cilja isti HTML element. Kaskada zapravo predstavlja niz filtara kroz koje naredbe i deklaracije prolaze kako bi se odredila

konačna vrijednost svakog pojedinog svojstva. Ovaj proces sprječava neodređenost u prikazu i omogućuje razvojnom programeru preciznu kontrolu nad stiliziranjem složenih dokumenata..

2.2.2. Sintaksa i struktura CSS-a

Sintaksa CSS-a (engl. Cascading Style Sheets) strukturirana je kao skup pravila (engl. rulesets) kojima se definira vizualna prezentacija elemenata. Svako pravilo čine dva osnovna dijela: selektor i deklaracijski blok. Selektor identificira HTML elemente na koje se pravilo odnosi, dok deklaracijski blok, smješten unutar vitičastih zagrada, sadrži jednu ili više deklaracija. Svaka deklaracija predstavlja par svojstvo: vrijednost, pri čemu se parovi razdvajaju dvotočkom, a pojedinačne deklaracije završavaju točkom-zarezom. Primjer osnovne sintakse koja definira tipografiju i boju teksta:

```
h1 { color: #333333; /* Boja teksta */
font-size: 24px; /* Veličina fonta */
text-align: center; /* Poravnanje teksta */
}
```

```
h1 { color: #333333; /* Boja teksta */
font-size: 24px; /* Veličina fonta */
text-align: center; /* Poravnanje teksta */
}
```

Slika 1- Primjer CSS koda

Budući da se na isti element može odnositi više različitih pravila, preglednik koristi mehanizam ***specifičnosti*** (engl. *specificity*) kako bi odredio koje pravilo ima prioritet u slučaju sukoba. Specifičnost se izračunava prema strogoj hijerarhiji selektora: prvi po prioritetu dolaze inline stilovi, zatim identifikatori, pa klase, atributi i pseudoklase i naposljetku elementi i pseodoelementi.

Temelj prostornog uređenja unutar ovog jezika predstavlja ***Box model***, koncept prema kojem se svaki HTML element interpretira kao pravokutna kutija sačinjena od četiri sloja. Središnji dio čini sam sadržaj, oko kojeg se nalazi unutarnji razmak (*padding*), potom granica (*border*) te na kraju vanjski razmak (*margin*) koji definira udaljenost elementa od ostatka sadržaja. Razumijevanje ovog modela, uz primjenu svojstva box-sizing: border-box, ključno je za precizno dimenzioniranje elemenata i postizanje predvidljivog rasporeda unutar dokumenta.

2.3. JavaScript

2.3.1. Općenito o JavaScriptu

JavaScript predstavlja programski jezik visoke razine koji podržava nekoliko različitih stilova programiranja te, uz HTML i CSS, čini treći najvažniji jezik suvremenog mrežnog razvoja i programiranja. Jezik je 1995. godine osmislio Brendan Eich za potrebe preglednika Netscape Navigator, a njegova je daljnja evolucija formalizirana kroz ECMAScript standard. Iako je izvorno bio namijenjen isključivo izvršavanju na klijentskoj strani, pojava okruženja Node.js omogućila je njegovu primjenu i na poslužiteljskoj strani, čime je JavaScript postao univerzalan alat za razvoj cjelovitih programskih rješenja.

Značajna prekretnica u razvoju jezika dogodila se 2015. godine objavom ES6 specifikacije (ECMAScript 2015), koja je uvela značajne promjene u sintaksu i funkcionalnost jezika, čime je kôd postao čitljiviji i lakši za održavanje. Poseban naglasak stavljen je na asinkrono programiranje uvođenjem sintakse, koja omogućuje učinkovito upravljanje mrežnim zahtjevima i dugotrajnim operacijama bez blokiranja glavne niti izvršavanja, što je od velike važnosti za učinkovitost suvremenih web stranica.

2.3.2. Sintaksa i struktura JavaScripta

Sintaksa JavaScripta u velikoj mjeri crpi inspiraciju iz programskog jezika C, koristeći vitičaste zagrade za definiranje blokova koda te točku-zarez na krajevima naredbi. Međutim, njezina unutarnja struktura počiva na specifičnim konceptima koji ovaj jezik čine izrazito fleksibilnim. Temeljna jedinica pohrane podataka su varijable, koje se deklariraju pomoću ključnih riječi let i const. Dok let omogućuje izmjenu vrijednosti unutar blokovskog opsega (engl. *block scope*), const definira konstante čija referenca ostaje nepromijenjena, čime se potiče stabilnost i predvidljivost programskog koda.

Logička struktura jezika oslanja se na izraze i operatore koji omogućuju manipulaciju podacima, dok kontrolne strukture poput uvjetnih grananja i petlji upravljaju tokom izvršavanja aplikacije. JavaScript omogućuje implementaciju naprednih tehnika poput funkcija višeg reda i zatvaranja (engl. *closures*), što je ključno za funkcionalni pristup programiranju.

```
const naslov = "Sustav";
let brojac = 0;

function provjeriStatus(vrijednost) {
    if (vrijednost > 0) {
        return naslov + ": Aktivno";
    } else {
        return naslov + ": Neaktivno";
    }
}

console.log(provjeriStatus(brojac));
```

Slika 2- Primjer JavaScript koda

Osim proceduralnih elemenata, struktura JavaScripta duboko je povezana s objektnim modelom programiranja. JavaScript koristi asinkroni model izvršavanja temeljen na petlji događaja (engl. *event loop*). Ovaj mehanizam omogućuje jeziku da upravlja višestrukim operacijama, poput korisničkih interakcija ili mrežnih zahtjeva, koristeći samo jednu nit izvršavanja. Time se osigurava da aplikacija ostane responzivna čak i tijekom obrade zahtjevnih pozadinskih zadataka, što predstavlja osnovu interaktivnosti na webu.

2.3.3. JSON (JavaScript Object Notation)

JSON (JavaScript Object Notation) lagan je format za razmjenu podataka koji je dizajniran kako bi bio čitljiv za ljude i jednostavan za strojnu obradu. Nastao je iz sintakse JavaScript objekata, ali je danas jezik-neovisan i podržan u gotovo svim programskim jezicima. JSON datoteke imaju ekstenziju .json i sastoje se od ključ-vrijednost parova, čime se postiže jasna i logična hijerarhija informacija.

Sintaksa JSON-a strogo je definirana kako bi se osigurala integritet i uniformnost podataka. Ključevi moraju biti znakovni nizovi u dvostrukim navodnicima, vrijednosti mogu biti znakovni nizovi, brojevi, logičke i null vrijednosti, objekti ili nizovi. Za razliku od standardnih JavaScript objekata, JSON format ne podržava funkcije, komentare niti nedefiniranim vrijednosti, što ga čini isključivo podatkovnim formatom bez programske logike. Ovakva restriktivnost osigurava maksimalnu kompatibilnost i sigurnost prilikom prijenosa informacija između različitih sustava.

2.4. Korišteni razvojni alati

2.4.1. Visual studio code

Visual Studio Code (VS Code) predstavlja suvremeno, lagano i visokoproduktivno okruženje za razvoj izvornoga kôda (engl. *source-code editor*), koje je razvila korporacija Microsoft. Iako se temelji na otvorenim tehnologijama, alat se profilirao kao industrijski standard zahvaljujući svojoj brzini, stabilnosti te iznimnoj prilagodljivosti potrebama razvojnih programera. Za razliku od ogrmonih integriranih razvojnih okruženja , VS Code nudi optimalanu ravnotežu između performansi i funkcionalnosti, što ga čini idealnim za razvoj mrežnih aplikacija utemeljenih na različitim jezicima.

Arhitektura alata počiva na modularnosti, što omogućuje proširivanje osnovnih značajki putem bogatih dodataka (engl. *extensions*). Ključne funkcionalnosti koje VS Code izdvajaju od sličnih uređivača teksta uključuju alate za ispravljanje pogrešaka, ugrađeni terminal koji omogućuje izvođenje skripti te IntelliSense što je napredna značajka koja nudi inteligentno dopunjavanje koda.

Slika 3-Visual studio code logotip

2.4.2. GitHub

GitHub je vodeća platforma zasnovana na oblaku (eng. cloud) koja služi za pohranu, upravljanje i kontrolu verzija izvornoga koda koristeći sustav Git. Iako Git omogućuje lokalno praćenje promjena na datotekama, GitHub pruža centraliziranu infrastrukturu koja olakšava suradnju, sigurnosnu pohranu i distribuciju programskih rješenja. Postao je nezaobilazan alat u sustavu mrežnog razvoja, omogućujući razvojnim programerima sinkronizaciju rada između različitih radnih stanica te transparentno praćenje povijesti razvoja projekta.

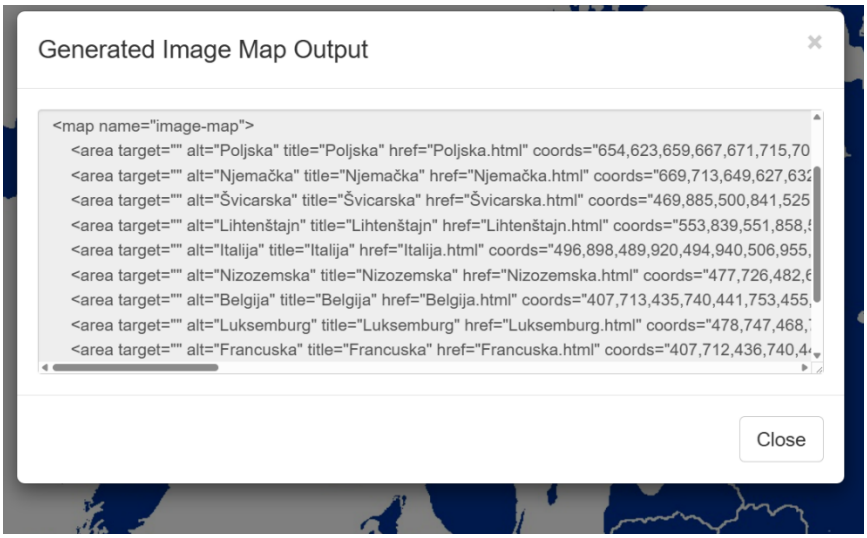
Slika 4-GitHub logotip

2.4.3. Image Map Generator

Image Map Generator predstavlja specijalizirani web alat namijenjen preciznom definiranju koordinata unutar HTML <map> elementa. Ja sam osobno koristio image-map. Osnovna namjena ovog alata je transformacija statične rasterske slike u interaktivno sučelje s više aktivnih polja (engl. *hotspots*). Ručno određivanje koordinata za složene oblike, poput granica država ili nepravilnih geometrijskih tijela, izrazito je zahtjevan i podložan pogreškama proces, stoga primjena ovakvog alata osigurava visoku razinu preciznosti i učinkovitosti u razvoju.

Slika 5-Iscrtavanje koordinata u Image Map generatoru

Alat na temelju vizualnog označavanja automatski generira HTML kôd koji sadrži <area> oznake s pripadajućim coord i shape atributima. Takav pristup omogućuje razvojnom programeru da se fokusira na logički dio aplikacije, dok se tehnička izmjera vizualnih elemenata automatizira, čime se postiže besprijekorna preciznost interakcije korisnika s grafičkim sučeljem.



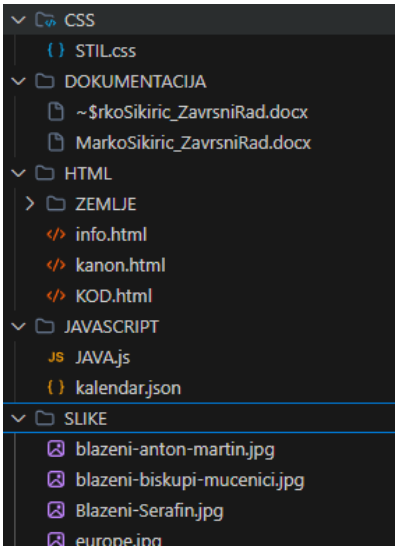
Slika 6-Automatski generiran kod za HTML primjenu

3. Tehničko-tehnološki dio

3.1. Struktura projekta

3.1.1. Općenita struktura projekta

Projekt je organiziran u jasnu hijerarhijsku strukturu mapa i datoteka. Na najvišoj razini nalaze se pet mapa: HTML (sve HTML stranice i podmapa ZEMLJE), CSS (stilska datoteka), JAVASCRIPT (JavaScript datoteka i Json), SLIKE (sve slike korištene na stranici) te DOKUMETACIJA(sama obrana završnog rada). Ovakav pristup razdvajanja sadržaja po vrsti olakšava snalaženje i održavanje projekta.



Slika 7-Struktura cijelog rada

Ovakva organizacija slijedi načelo razdvajanja dužnosti (separation of concerns) gdje svaka vrsta datoteke ima svoju mapu i ne miješa se s ostalima. HTML datoteke sadrže strukturu, CSS stilove, a JavaScript logiku. Slike su centralizirane u jednoj mapi radi lakšeg upravljanja.

Ukupno projekt sadrži: 3 glavne HTML stranice KOD.html koji sadrži i samu interaktivnu mapu, kanon.html koji sadrži informacije o svecima te ispisuje svece dana i info.html koji sadrži informacije o izradi. 46 HTML stranica zemalja nalaze se unutar mape ZEMLJE, 1 CSS datoteku (STIL.css), 1 JavaScript datoteku (JAVA.js), 1 JSON datoteku (kalendar.json) i oko 40 fotografija svetaca u mapi SLIKE, te konačno i word dokument same dokumentacije završnog rada.

3.1.2. Glavna stranica s mapom

Datoteka KOD.html predstavlja središnju komponentu projekta i služi kao polazišna točka koja definira temeljnu strukturu cijele WEB STRANICE. Na samom početku datoteke nalazi se deklaracija tipa dokumenta te zaglavlje u kojemu su definirani ključni meta-podaci, naslov stranice te poveznice prema vanjskim datotekama stilova. Neposredno nakon toga slijedi element header koji sadrži zaglavlje stranice, dok navigacijska traka implementirana unutar elementa nav omogućuje intuitivno kretanje kroz stranicu te se na samom kraju nalazi element footer s informacijama o autorskim pravima i popratnim podacima projekta. Ovaj dio je potpuno isti za svaku html stranicu, dok se razlikuju jedino središnji funkcionalni blokovi dokumenta. U KOD.html rezerviran je za interaktivnu sliku karte europskih zemalja.

Interaktivna karta Europe izvedena je primjenom HTML Image Map mehanizma unutar specijaliziranog spremnika. Glavni vizualni element čini rasterska slika karte koja je putem atributa usemap povezana s odgovarajućim elementom map. Unutar te strukture definirano je 47 preciznih polja za interakciju, pri čemu svako polje predstavlja zasebnu državu. Svako je polje definirano s oblikom (poly), vlastitim koordinatama, poveznicom na html datoteku svake države te elementa za prikaz obavijesti, koji služi za dinamičko ispisivanje naziva država prilikom prelaska mišem preko površine te države.

```
<div class="map-container">
  <img src="..\SLIKE\europe.jpg" usemap="#mapa-europe" id="mapa-slike" alt="Mapa Europe">
  <div id="tooltip"></div>

  <map name="mapa-europe">

    <area shape="poly" coords="650,913,686,904,716,866,772,890,779,909,704,918,772,994,714,967,692,948"
      href="ZEMLJE/hrvatska.html" alt="Hrvatska" title="Hrvatska" data-name="Hrvatska">
```

Slika 8-Primjer koda središnjeg dijela KOD.html datoteke

Završni dio dokumenta, smješten neposredno prije zatvaranja tijela stranice, obuhvaća uključivanje vanjskih skriptnih datoteka. Sustav koristi vanjsku imageMapResizer biblioteku koja osigurava odzivnost slikovne mape za različite veličine zaslona, te lokalnu datoteku JAVA.js u kojoj se nalazi logika web stranice.

3.1.3. Datoteke zemalja

Za svaku od 46 europskih država prikazanih na interaktivnoj karti izrađena je zasebna HTML datoteka smještena unutar podmape ZEMLJE, čime je postignuta pregledna organizacija datoteka mnogobrojnih zemalja. Zaglavlje, navigacija te podnožje su jednaki kao

i početni i glavni KOD.html, jedino se razlikuje središnji blog koji u stranicama zemalja sadržava naslov države te kratki opis sveca iz te države uz njegovu sliku.

Kartica sveca koristi klasu `<div class="sveci">` sa slikom sveca s `float: right` pozicioniranjem te podnaslov `<h2>` s imenom, te konačno i odlomke <p> s biografskim tekstom koji je najznačajniji za svakog pojedinog sveca.

```
<h1>HRVATSKI SVECI</h1>

<div class="sveci">
  <img src="../../SLIKE/sveti-nikola.jpg" alt="">
  <h2>Sveti Nikola Tavelić</h2>
  <p> Rođen je oko 1340. godine u Šibeniku. Kao mladi
    propovijeda bogumilima u Bosni, a zatim s trojicom sub
    Kurban Bajrama, otišli su do čuvene Omarove džamije i
    ih
```

Slika 9-Primjer koda za html pojedine zemlje

3.1.4. Stranica Kanonizacija (kanon.html)

Datoteka kanon.html služi kao informativno središte projekta, a njezin je sadržaj organiziran kroz šest tematskih cjelina unutar spremnika klase kanonizacija. Prvih pet odlomka detaljno opisuju korake procesa ka proglašenju svetim u Katoličkoj crkvi, koristeći dvojezičnu terminologiju. Naslovi su istaknuti na latinskom jeziku u kurzivu (*italic*), uz pripadajući prijevod na hrvatski jezik, radi prilagodbe crkvenim standardima.

Uz navedeni statični sadržaj, šesti element iste klase služi kao prostor za integraciju dinamičkih informacija. Unutar ovog segmenta nalazi se identifikator svetac-dana, koji funkcionira kao spremnik za podatke koji se generiraju prilikom učitavanja stranice. Programska logika, sadržana u datoteci JAVA.js, automatski popunjava ovaj element relevantnim tekstualnim sadržajem, čime se statični edukativni elementi nadopunjuju informacijama u stvarnom vremenu.

```
<div class="kanonizacija" style="text-align: center;">
    <h2>Sveci danas</h2>
    <p>Danas slavimo: <span id="svetac-dana">Učitavam...</span></p>
</div>

<script src="..\JAVASCRIPT\JAVA.js"></script>
```

Slika 10-Kod dinamičkog elementa stranice Kanonizacija

Prikazani isječak koda prikazuje povezivanje HTML strukture s programskom logikom. Unutar spremnika klase kanonizacija nalazi se element `<span id="svetac-dana">` koji sadrži privremeni tekst „Učitavam...". Taj tekst služi samo kao zamjena za prave informacije, te je vidljiv samo dok JavaScript ne završi obradu podataka.

Upotreba jedinstvenog identifikatora id omogućuje skripti precizno lociranje elementa unutar DOM strukture. Vanjska datoteka JAVA.js, uključena putem oznake <script>, te upravo ona inicijalizira dinamičku zamjenu teksta stvarnim podacima. Ovakav pristup razdvaja prezentacijski sloj od logičkog, osiguravajući da se sadržaj ažurira automatski po učitavanju stranice, što poboljšava korisničko iskustvo i povećava interaktivnost.

3.1.4.1. Datoteka kalendar.json – baza podataka

Datoteka kalendar.json smještena je u mapi JAVASCRIPT i predstavlja bazu podataka godišnjeg crkvenog kalendara. Datoteka sadrži 366 unosa koji pokrivaju sve dane u godini uključujući 29. veljače za prijestupne godine. Tehnički je svaka stavka izvedena kao mapiranje specifičnog datuma u formatu MM-DD na pripadajući tekstualni opis blagdana ili svetca koji se obilježava toga dana..

Podaci su organizirani kao jedan JSON u kojem ključeve čine kronološki poredani datumi. Vrijednosti pridružene tim ključevima su tekstualni nizovi koji ponekad sadrže više svetaca odvojenih točkom-zarezom. Blagdani su ispisani velikim slovima radi isticanja, dok su spomendani svetaca navedeni standardnim načinom pisanja. Ovakva organizacija omogućuje JavaScript logici brzo i učinkovito pretraživanje baze te izravno dohvaćanje relevantnih informacija bez potrebe za kompleksnim upitima, što izravno doprinosi brzini rada korisničkog sučelja.

Datum	Vrijednost u kalendar.json
"01-01":	"NOVA GOD.; MARIJA BOGORODICA",
"01-02":	"Bazilije Veliki; Grgur Nazijanski",
"01-03":	"Ime Isusovo, Mladenka",
"01-04":	"2. po Božiću. KROZ GOD. Anđela; Borislav; Emanuel",
"01-05":	"Miljenko; Emilijana; Radoslav",

Tablica 1-Primjeri unosa datoteke kalendar.json

3.1.5. Stranica O izradi (info.html)

Stranica info.html sadrži dva informacijska panela. U prvom je opisana osobna motivacija za projekt poput vlastite inspiracija i cilj projekta. Citat svetog Antuna Padovanskog stiliziran je italik formatom unutar teksta.

Drugi panel prikazuje meta-podatke o završnom radu: ime autora i mentora, razred , naziv škole,tei poveznicu na dokumentaciju projekta. Ova stranica ispunjava i praktičnu svrhu transparentnosti gdje svaki posjetitelj može saznati tko stoji iza projekta i koji je izravni cilj i razlog projekta.

3.1.6. Datoteka STIL.css – stilovi svih elemenata

3.1.6.1. Stil tijela stranice

Globalni stil body elementa temelji se na fontu Garamond, čija serifna estetika odgovara povijesnoj tematici rada, te boji teksta #a84a4a koja evocira osjećaj topline i tradiciju. Pozadina je definirana slikom pozadina.jpg uz svojstvo background-attachment: fixed. Time je postignut parallax efekt pri kojem pozadina ostaje statična, dok se sadržaj pomiče, što sučelju daje dubinu i moderan vizualni dojam.

```
body {
    font-family: garamond, sans-serif;
    color: #a84a4a;
    background-image: url('../SLIKE/pozadina.jpg');
    background-size: cover;
    background-attachment: fixed;
}
```

3.1.6.2. Stil navigacijske trake

Navigacijska traka implementira moderni glassmorphism efekt što je dizajnerski trend koji oponaša izgled mat stakla. Postiže se kombinacijom backdrop-filter: blurkoji zamagljuje pozadinu iza elementa i background: rgba(255,255,255,0.7) koji dodaje polutransparentnu bijelu boju. CSS svojstvo position: sticky uz top: 0 osigurava da traka ostaje trajno vidljiva pri stalnom pomicanju stranice.

Gumbi navigacije stiliziraju se s crvenkastom pozadinom (#a84a4a), bijelim tekstom i zaobljenim kutovima (border-radius: 8px). Hover stanje dodaje podizanje gumba transformacijom i produbljenjem sjene. Aktivna stranica označena je zlatno-žutom bojom gumba (rgba(179,179,60,0.9)) čime se postiže lakša organiziranost korisnika.

```
nav {
    position: sticky;
    top: 0;
    z-index: 1000;
    backdrop-filter: blur(12px);
    background: rgba(255, 255, 255, 0.7);
    box-shadow: 0 1px 10px rgba(0, 0, 0, 0.05);
}
```

3.1.6.3. Kartica sveca

CSS klasa .sveci stilizira bijele panele s informacijama o svecima. Panel ima zaobljene kutove (border-radius: 10px), višeslojnu sjenu (box-shadow: 0 4px 8px rgba(0,0,0,0.1)) i maksimalnu širinu od 800 piksela centriranu na stranici (margin: 40px auto).

Slika sveca unutar kartice (.sveci img) je centrirana desno (float: right) i definiranom širinom te omogućuje tekstu da se prilagodi oko slike s lijeve strane te ispod slike. Svojstvo mix-blend-mode: normal izričito prekida globalnu stilsku deklaraciju mix-blend-mode: multiply koja vrijedi za sve ostale slike, osiguravajući prirodan prikaz fotografija svetaca.

```
.sveci {
    background-color: rgba(255, 255, 255, 0.9);
    padding: 20px;
    margin: 40px auto;
    max-width: 800px;
    border-radius: 10px;
    text-align: left;
    overflow: auto;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}

.sveci img {
    float: right;
    width: 250px;
    height: auto;
    margin-left: 20px;
    border-radius: 8px;
    mix-blend-mode: normal;
}
```

Slika 11- CSS kod za prikaz uređenja svetaca

3.1.6.4. Podnožje (.detailed-footer)

Podnožje stranice prati vizualni koncept glassmorphism dizajna, čime se postiže estetska usklađenost s navigacijskom trakom. Za raspored elemenata korišten je Flexbox model sa svojstvima flex-wrap: wrap i justify-content: space-between, što

omogućuje automatsku organizaciju sadržaja u tri stupca. Ovakav pristup osigurava preglednost i respozivnost, jer se pri smanjenju širine zaslona stupci prirodno postavljaju u redove. Svaka sekcija unutar podnožja definirana je minimalnom širinom od 200 piksela, dok su podnaslovi vizualno istaknuti crvenkastom podvučenom linijom, čime je zadržana dosljednost s ostatkom vizualnog identiteta aplikacije.

3.1.7. Datoteka JAVA.js – JavaScript funkcionalnosti

3.1.7.1. Dinamički prikaz svetaca dana

Asinkrona funkcija ucitajSveca() ključni je dio programske logike koji povezuje bazu podataka u JSON formatu s korisničkim sučeljem. Njezino izvršavanje započinje nakon potpunog učitavanja DOM strukture dokumenta, što je osigurano slušačem događaja DOMContentLoaded. Koristeći ugrađeni objekt Date, sustav dohvaća trenutni mjesec i dan. Primjenom metode padStart(2, '0') osigurava se da jednoznamenkasti brojevi dobiju vodeću nulu, čime se generira format MM-DD koji odgovara ključevima u JSON datoteci. Asinkrono dohvaćanje (Fetch API) je funkcija koja šalje mrežni zahtjev prema datoteci kalendar.json. Korištenje ključne riječi await omogućuje da se izvršavanje skripte privremeno zaustavi dok se podaci u pozadini ne dohvate i ne pretvore u obradiv JavaScript objekt pomoću metode .json(). Iz dobivenog objekta izravno se dohvaća vrijednost pridružena današnjem datumu te se ona metodom innerHTML upisuje u predviđeni element na stranici. Ovim mehanizmom postignuta je dinamička prilagodba kalendarskom vremenu svakog pojedinog korisnika.

```javascript
document.addEventListener('DOMContentLoaded', ucitajSveca);

async function ucitajSveca() {
    // 1. Dohvati današnji datum u formatu MM-DD
    const danas = new Date();
    const mjesec = String(danas.getMonth() + 1).padStart(2, '0');
    const dan = String(danas.getDate()).padStart(2, '0');
    const formatDatuma = `${mjesec}-${dan}`;

    // 2. Povuci JSON datoteku i pretvori u objekt
    const odgovor = await fetch('../JAVASCRIPT/kalendar.json');
    const podaci = await odgovor.json();

    // 3. Pronađi sveca i upiši ga u HTML
    document.getElementById('svetac-dana').innerHTML = podaci[formatDatuma];
}

document.addEventListener('DOMContentLoaded', ucitajSveca);
```

Slika 12-Dinamičko dohvačanje sveca dana

3.1.7.2. Interaktivnost mape Europe

Programska podrška za interaktivnu kartu Europe temelji se na dvije funkcionalne cjeline unutar JAVA.js datoteke. Prvi segment osigurava responzivnost sustava korištenjem vanjske biblioteke imageMapResizer. Pozivom funkcije imageMapResize() neposredno nakon učitavanja DOM-a, skripta automatski skalira koordinate elemenata <area> u skladu s trenutnim dimenzijama slike. Ovaj mehanizam jamči preciznost klikabilnih polja neovisno o veličini zaslona korisnika ili naknadnim promjenama veličine prozora preglednika.

Druga cjelina implementira sustav za prikaz naziva država u stvarnom vremenu (**tooltip**). Logika se temelji na iteraciji kroz sve elemente <area> te registraciji dvaju ključnih slušača događaja: **mousemove:** Prilikom pomicanja kursora iznad polja države, funkcija dinamički dohvaća naziv iz atributa data-name. Istovremeno se vrši izračun pozicije pomoću metode getBoundingClientRect(), koja omogućuje precizno pozicioniranje obavijesti relativno u odnosu na kontejner karte, umjesto na cijeli prozor. Rezultat je vizualno ugodan okvir koji prati kretanje miša uz fiksni odmak (*offset*). **mouseleave:** Čim kursor napusti poligon države, CSS svojstvo display postavlja se na none, čime se element skriva s površine sučelja.

Ovakva implementacija omogućuje fluidnu navigaciju i pruža trenutačnu informaciju korisniku, čime se poboljšava intuitivnost i funkcionalnost interaktivnog zemljovida.

4. Zaključak

Ovim završnim radom izrađena je potpuna, javno dostupna i funkcionalna web stranica koja na interaktivan i vizualno dosljedan način prikazuje europske svece. Korištenjem isključivo temeljnih web tehnologija poput HTML-a, CSS-a i JavaScripta demonstrirano je izvrsno razumijevanje svih triju slojeva modernog front-end web razvoja.

Tehnički gledano, projekt uspješno integrira nekoliko različitih tehnika i koncepta. HTML Image Map s poligonalnih zona omogućuje intuitivnu interakciju s kartom Europe. CSS glassmorphism efekt, parallax pozadina i Flexbox rasporedi daju stranici moderan vizualni identitet koji nije zastario unatoč korištenju samo CSS-a. JavaScript asinkrono dohvaćanje JSON datoteke putem Fetch API-ja i dinamički prikaz sveca dana uvode dinamičnost bez potrebe za serverom ili bazom podataka u klasičnom smislu. Također objava putem GitHub Pages dokazuje da statični web projekt može biti javno dostupan besplatno i bez složene serverske infrastrukture.

Zaključno, ovaj projekt predstavlja uspješan spoj gradiva naučenog tijekom srednjoškolskog obrazovanja i osobne humanističke svrhe. Spojena vjera i digitalna tehnologija, također i ukazuju na moje daljnje visoko obrazovanje na Hrvatskom Katoličom Sveučilištu na smjeru Računalstvo za računalne primjene koje spaja upravo to, vjeru i informacijske tehnologije. U sadašnjem obliku projekt ispunjava svoju temeljnu svrhu da modernom tehnologijom očuva i prenese bogatu duhovnu baštinu europskih naroda novim generacijama te ju učiniti pristupačnijom, razumljivijom i prilagođenom potrebama suvremenog korisnika u digitalnom dobu.

Popis literature

[1] Mozilla Developer Network. HTML: HyperText Markup Language. URL: https://developer.mozilla.org/en-US/docs/Web/HTML, (pristupljeno: travanj 2026.)

[2] Mozilla Developer Network. CSS: Cascading Style Sheets. URL: https://developer.mozilla.org/en-US/docs/Web/CSS, (pristupljeno: travanj 2026.)

[3] Mozilla Developer Network. JavaScript. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript, (pristupljeno: travanj 2026.)

[4] Mozilla Developer Network. Fetch API. URL: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API, (pristupljeno: travanj 2026.)

[5] David J. Bradshaw. imageMapResizer – GitHub repozitorij. URL: https://github.com/davidjbradshaw/image-map-resizer, (pristupljeno: travanj 2026.)

[6] Ecma International. ECMAScript 2015 Language Specification (ES6). URL: https://www.ecma-international.org/ecma-262/, (pristupljeno: travanj 2026.)

[7] ECMA International. The JSON Data Interchange Syntax. URL: https://www.json.org, (pristupljeno: travanj 2026.)

[8] GitHub Inc. GitHub Pages dokumentacija. URL: https://docs.github.com/en/pages, (pristupljeno: travanj 2026.)

[9] Microsoft Corporation. Visual Studio Code dokumentacija. URL: https://code.visualstudio.com/docs, (pristupljeno: travanj 2026.)

[10] Image Map Generator. URL: https://www.image-map.net, (pristupljeno: travanj 2026.)

[13] W3Schools. HTML Image Maps. URL: https://www.w3schools.com/html/html_images_imagemap.asp, (pristupljeno: travanj 2026.)

Popis Slika

Slika 1- Primjer CSS koda 4

Slika 2- Primjer JavaScript koda 6

Slika 3-Visual studio code logotip 7

Slika 4-GitHub logotip 8

Slika 5-Iscrtavanje koordinata u Image Map generatoru 9

Slika 6-Automatski generiran kod za HTML primjenu 10

Slika 7-Struktura cijelog rada 11

Slika 8-Primjer koda središnjeg dijela KOD.html datoteke 12

Slika 9-Primjer koda za html pojedine zemlje 13

Slika 10-Kod dinamičkog elementa stranice Kanonizacija 13

Slika 11- CSS kod za prikaz uređenja svetaca 17

Slika 12-Dinamičko dohvačanje sveca dana 19

Popis tablica

Tablica 1-Primjeri unosa datoteke kalendar.json 15